

Node JS

Programmation JavaScript côté serveur

Mohamed Amine Ben Rhouma 2025-2026



Plan

○ **Prérequis :**

- Bonnes connaissances du langage Javascript. Une première approche d'un framework JavaScript (côté client) serait un plus.

○ **Objectifs / Compétences visées :** Apprendre à développer une application serveur avec Node.js :

- Connaître les fondements de fonctionnement de Node JS
- Installer et configurer un serveur Node.js
- Manipuler l'API de Node.js
- Gérer la persistance des données (éventuellement dans une base NoSQL)

○ **Méthodes pédagogiques :**

- Notions théoriques et notes de cours
- Démonstrations et activités pratiques

Plan de formation

1. Présentation générale
2. Les premiers pas Node JS
3. Express
4. Bases de données NoSQL : MongoDB
5. **Exemple d'application JS FullStack**



1.

Présentation générale

- ☐ Introduction
- ☐ C'est quoi Node JS ?
- ☐ Pourquoi Node JS ?
- ☐ Mode fonctionnement
- ☐ Installation



Introduction

Problème de latence

:

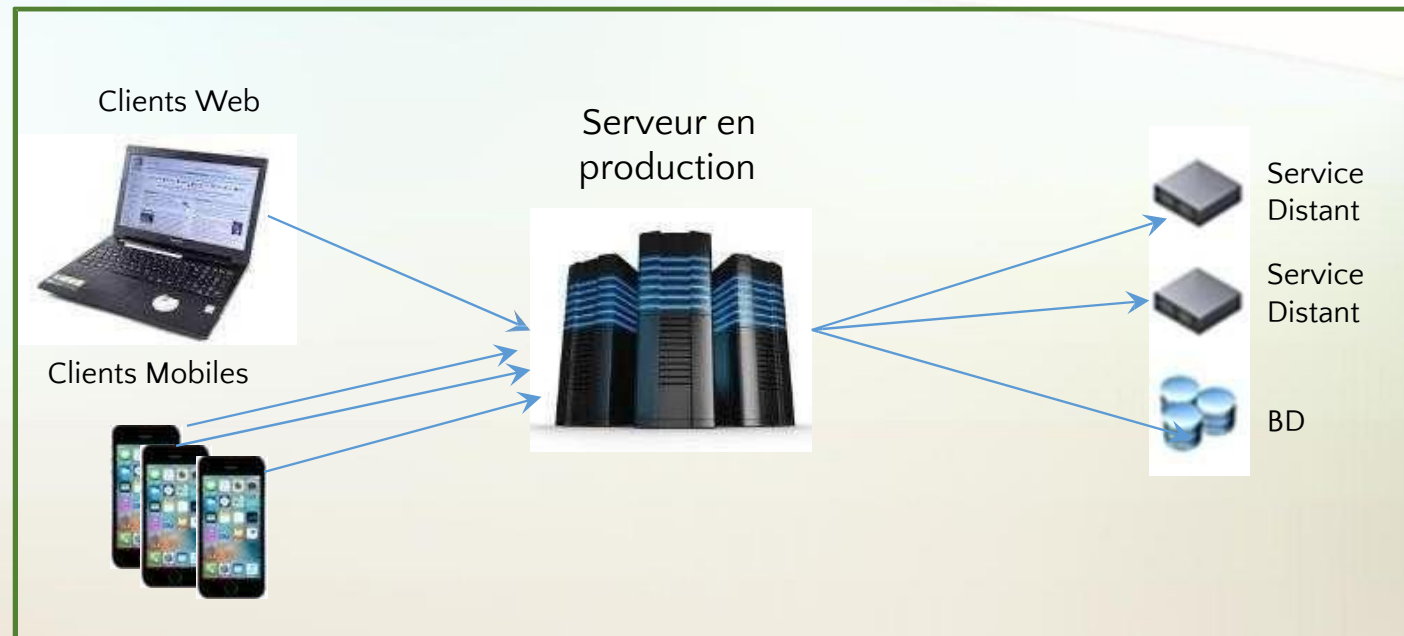
Les applications qui tournent en production :

- Une variété de clients
- une variété de services distants

Problème et contraintes :

- Des clients qui ont des connexions lentes (Long lived) et qui monopolisent des ressources sur le serveur
- Une API distante avec un problème de latence.

Ce qui peut ralentir le service (voir le rendre complètement indisponible)





Présentation générale

- **C'est quoi Node JS ?**
- Plateforme de développement : ensemble de librairies JavaScript
- Créée par Ryan Lienhart Dahl en 2010
- Open-Source, développé en C++ et Cross-plateforme
- Permettant l'exécution de JavaScript côté serveur et de réaliser des tâches comme :
 - Persistance de données
 - Manipulation de fichiers
 - Sécurité, ...
- Utilise le moteur d'exécution V8 de Google Chrome



Présentation générale

- **Pourquoi Node JS** Bien adapté pour le « back-end »
 - ❏ **Utilisation de JavaScript** : Facile à apprendre surtout pour ceux qui connaissent déjà JavaScript : réduire le cout de développement surtout que les applications frontEnd et backEnd peuvent partager du code JS
 - ❏ **Très rapide** : grâce à la version 8 du moteur développé par Google qui compile le JavaScript en code machine natif et fonctionne à la vitesse de l'éclair
 - ❏ **Flexible** : Grâce au NPM intégré, les développeurs peuvent mettre à jour, partager ou réutiliser des codes avec une extrême facilité
 - ❏ **Populaire** : technologie appréciée par de plus en plus d'entreprises. Plusieurs géants du web, dont Microsoft, Google, Yahoo, Mozilla et Github, l'ont même adopté.



Présentation générale : JavaScript VS Node JS

- **JavaScript**

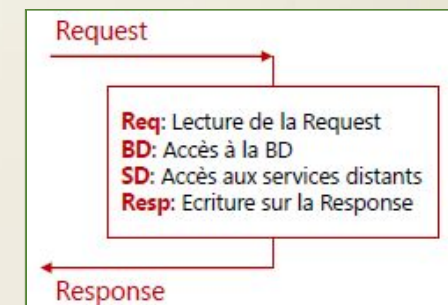
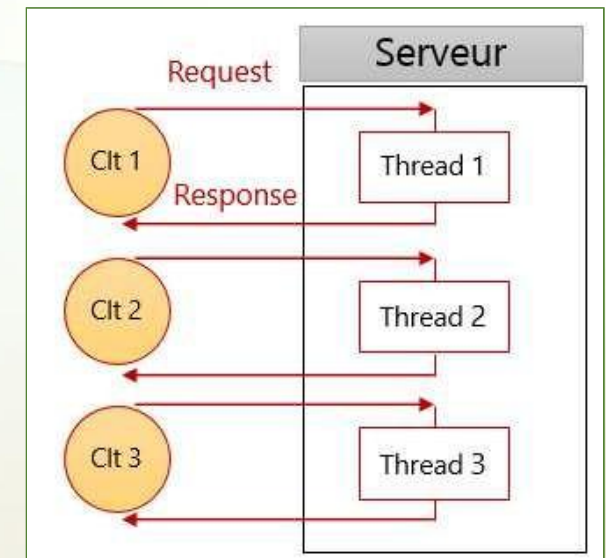
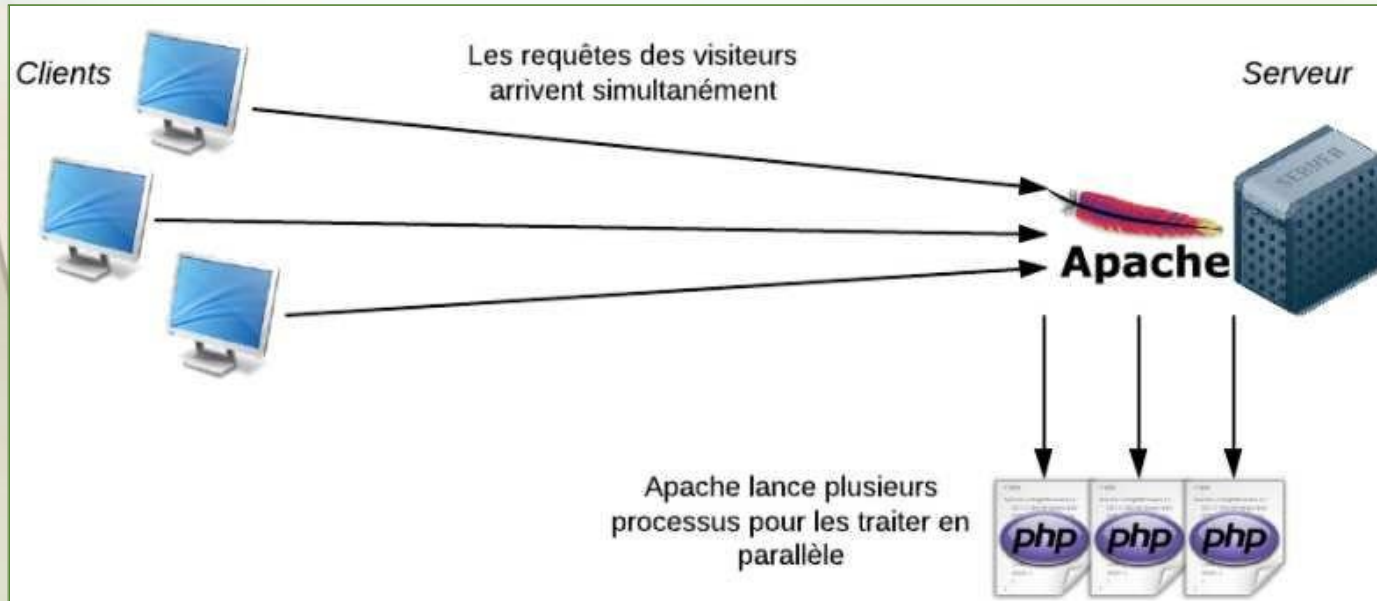
- permet d'écrire des scripts qui seront exécutés côté client
- Langage de programmation qui sera interprété par le moteur du navigateur pour obtenir une application utilisable par les visiteurs (affichage, modifications, animations, ...)

- **Node**

- Environnement permettant d'écrire du code JS qui s'exécute côté serveur : fonctionne en dehors du navigateur

Modèle Multi Threads Bloquant

- **Fonctionnement naturel de Apache/PHP :**
 - Multithreading
 - Exécutions synchrones de PHP
- Modèle classique Bloquant basé sur une Pool de Threads : à chaque requête, on affecte un Thread tiré du pool de centaines de threads.

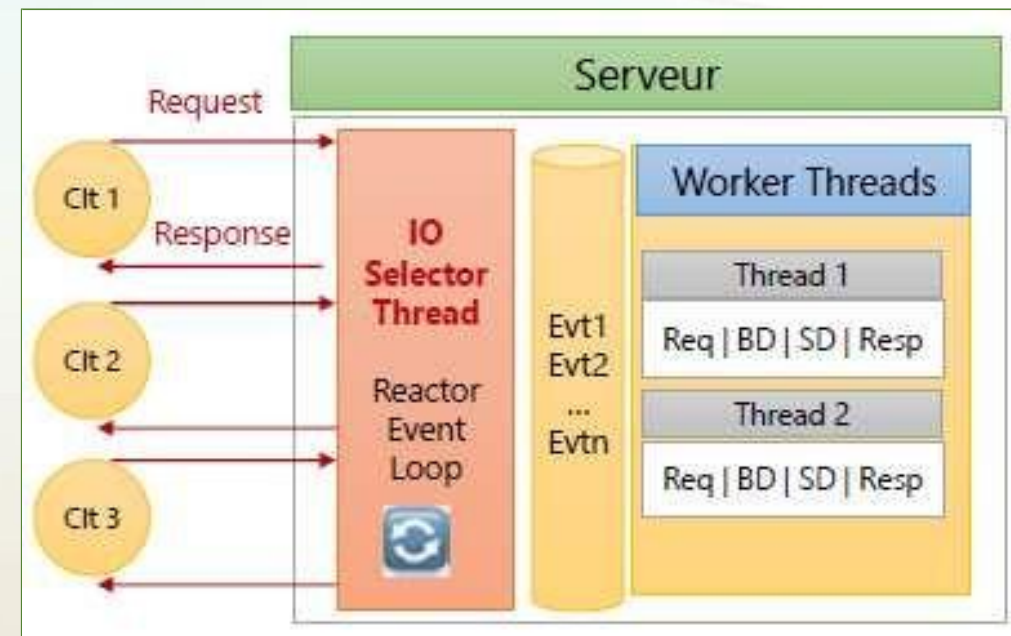




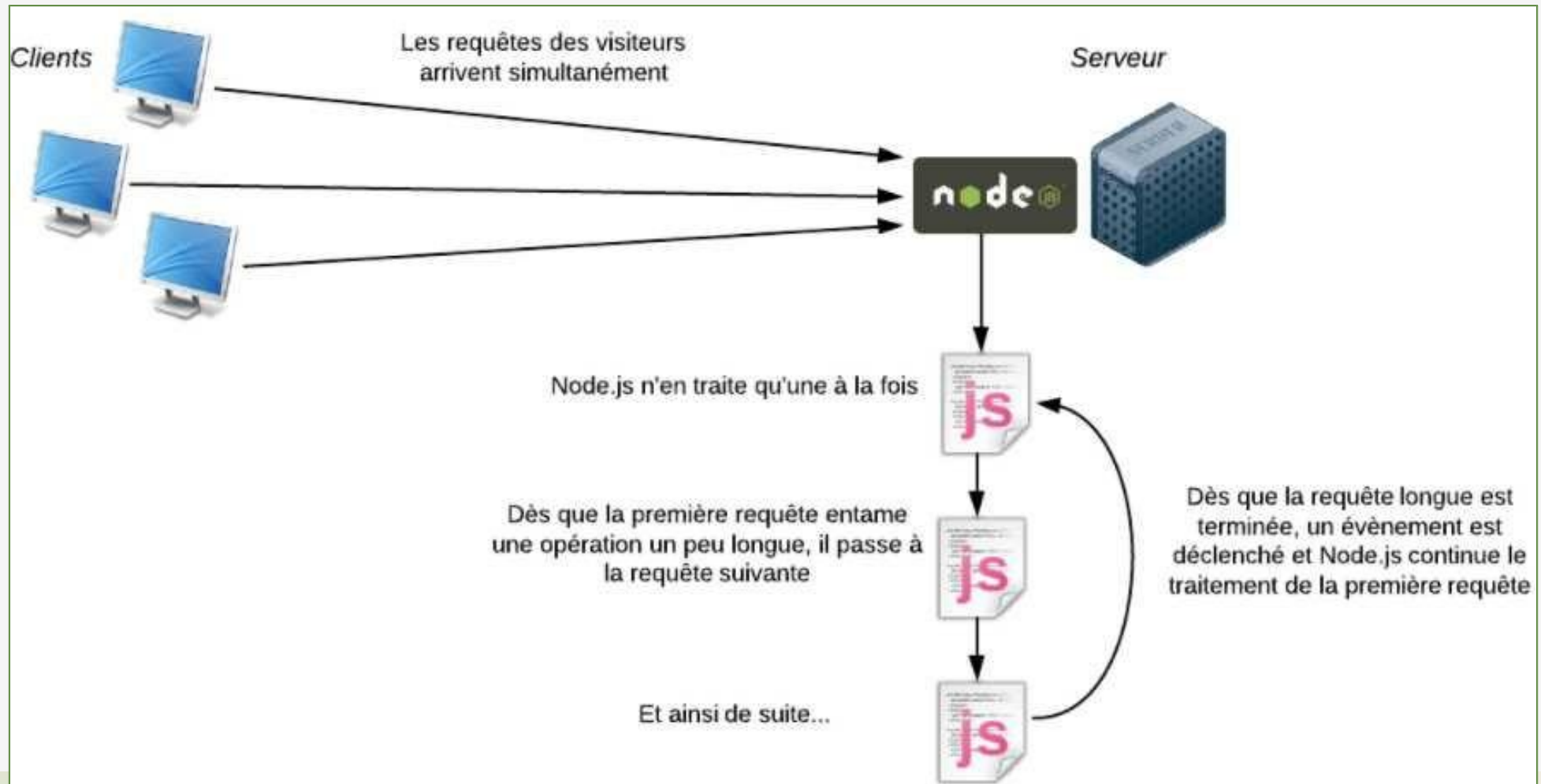
Node JS : Mode de fonctionnement

Modèle « Single Thread non bloquant »

- Un IO Selector Thread dont le rôle est d'orchestrer les entrées/sorties non bloquantes.
- Cette fois-ci tous les IO doivent être faits d'une manière **non bloquantes**. Ce qui fait qu'on va jamais attendre
- Cet IO thread va **gérer les lectures et les écritures comme des évènements** qu'il va empiler et dépiler dans une Queue d'une manière non bloquante.
- **Un nombre réduit de Worker Threads** (en fonction du nombre de CPU du serveur) : traiter les requêtes de manière non bloquante. Ils ne vont jamais attendre. Ils seront toujours en train de travailler et exploiter au maximum les ressources du serveur



Node JS : Mode de fonctionnement

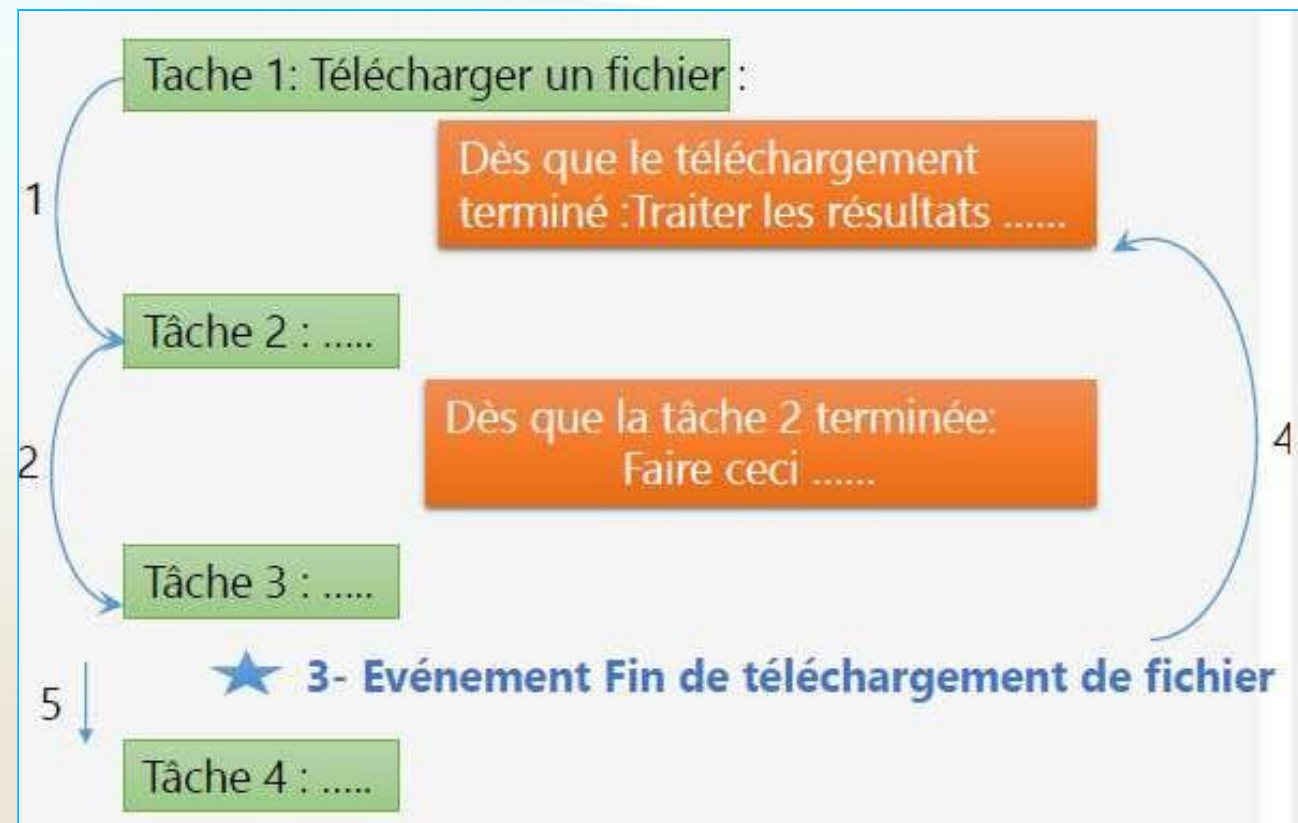




Node JS : modèle non bloquant

- **Node JS** : architecture de code entièrement non bloquante

- Le code NodeJS est asynchrone piloté par des événements et en utilisant des callbacks pour chaque action.
- Ce qui permet qu'avec un seul Thread, on peut traiter simultanément plusieurs requêtes.
- S'appuie sur V8, le moteur JavaScript de Google utilisé dans Chrome, qui fait partie des moteurs JavaScript les plus puissants.





Node JS : installation

- **Installation :**

- <https://nodejs.org/fr/download/>
- **node -v** : version de node

Téléchargements

Dernière version LTS: **14.17.4** (inclut npm 6.14.14)

Téléchargez le code source de Node.js pour votre système d'exploitation et commencez à développer dès aujourd'hui.

LTS Recommandé pour la plupart des utilisateurs	Dernière Dernières fonctionnalités	
 Installeur Windows node-v14.17.4-x64.msi	 Installeur macOS node-v14.17.4.pkg	 Code Source node-v14.17.4.tar.gz

- Après installation de NodeJS, vous disposez de :
 - L'outil **Node** qui permet d'exécuter une application NodeJS
 - L'outil **npm** (Node Package Manager) qui est un gestionnaire de paquets officiels de NodeJS.
 - Initialiser le projet
 - Installer les dépendances
 - Lancer des scripts
 -

Etc..

2.

Les premiers pas Node JS

- ☐ Initialisation du projet
- ☐ Les modules
- ☐ Les fonctions callback
- ☐ Les fonctions synchrones et asynchrones
- ☐ Création du serveur



Node JS : initialisation d'un projet

- Initialisation d'un projet NodeJS :

- `npm init`

ou encore `npm init -y` (sans le mode interactif)

- Permet de générer le fichier **package.json** :

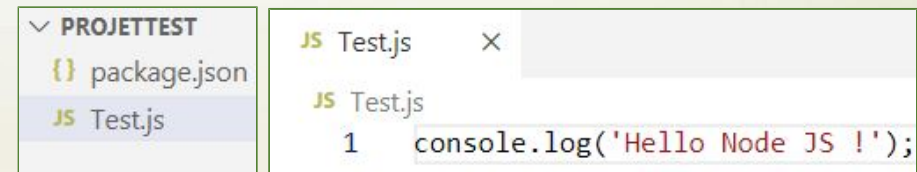
- les infos sur le projet,
 - déclarer les dépendances du projet,
 - déclarer les scripts,
 - Etc...

- Exécution d'un projet NodeJS :

- `node nomFichier`

```
> npm init
```

```
package name: (projettest)
version: (1.0.0)
description:
entry point: (Test.js)
test command:
git repository:
keywords:
author:
license: (ISC)
```





Les modules

- **C'est quoi un module ?**
 - Un fichier pouvant contenir des variables, constantes, fonctions, classes...
- **Propriétés :**
 - Il est possible d'utiliser des éléments définis dans un autre fichier
 - Pour cela, il faut l'importer là où on a besoin de l'utiliser
 - Pour importer un élément, il faut l'exporter dans le fichier source
- **Exports, require :**
 - Syntaxe native de NodeJS définie dans CommonJS : système de gestion de module de NodeJS.



Les modules

- **Trois types de modules :**

- Modules définis dans le noyau du nodeJS : pour les utiliser, il faut juste écrire :

`require('nomModule');`

- Modules de la communauté NodeJS : pour les utiliser, il faut les télécharger puis les utiliser via la console.
 - Ils sont gérés par NPM (Node Package Manager), voir www.npmjs.com
- Nos propres modules : pour les utiliser, il faut les exporter puis les importer avec **`require('./nomModule');`**



npm : Node Package Manager

- **Quelques commandes avec npm :**
 - `npm install nomModule` : Installer localement un module (ajoute généralement un répertoire `node_modules` dans le répertoire courant)
 - `npm install -g nomModule` : Installer globalement un module
 - `npm help nomCommande` : Avoir de l'aide sur une commande
 - `npm ls` : Lister les modules installés
 - `npm ls -g` : Lister les modules globaux installés
 - `npm root` : Localiser le `node_modules`
 - `npm init` : Initialiser un projet



npm : Node Package Manager

- Quelques commandes avec npm :
 - `npm search nomModule` : Chercher un module
 - `npm uninstall nomModule` : Désinstaller un module
 - `npm uninstall -g nomModule` : Désinstaller un module global
 - `npm link nomModule` : Linker un module global à un projet local
 - `npm outdated` : Lister les modules qui ne sont pas à jour
 - `npm update` : Mettre à jour les modules
 - `npm config` : Reconfigurer NPM



Exemples

- **Exemple 1 :**

- Dans un nouveau répertoire, initialiser le projet avec **npm init**
- Utiliser le module **os** pour afficher :
 - L'architecture, le nombre de CPU, le hostname et le répertoire courant.

- Code source :

```
var os = require("os");  
console.log("Architecture:", os.arch());  
console.log("Nombre CPU", os.cpus().length);  
console.log("Hostname", os.hostname());  
console.log("Répertoire", os.homedir());
```



Exemples

- **Exemple 2 :**
 - Avec la console, exécuter `npm install --save lodash`
`lodash` : bibliothèque JS incluant des fonctions utilitaires pour manipuler des données (tableaux, chaînes, objets, ...)
 - Vérifier l'ajout de `lodash` dans la section `dependencies` de `package.json`
 - Tester le code suivant :

```
var math = require('lodash');  
console.log(math.map([1, 5, 3], function (a) {  
    return a * 2  
}));
```



Exemples

- **Exemple 3 :**

- Créer le fichier : `mesModules.js`

```
var direBonjour = function () {  
    console.log('Bonjour');  
}  
module.exports = direBonjour;
```

- Utiliser la fonction `require` dans un autre fichier `test.js` pour importer les modules définis dans `mesModules.js`

```
var mod = require('./mesModules')  
mod()
```

NB : On peut exporter des fonctions, des constants, variables, objets, ...



Exemples

- **Exemple 3 : Une autre façon de faire**

- Au niveau du fichier : **mesModules.js**

```
exports.direBonjour = function () {  
    console.log('Bonjour'); }  
}
```

- Dans le fichier **test.js**:

```
var mod = require('./mesModules')  
mod.direBonjour()
```

- Ou encore avec la définition de deux fonction

```
:  
exports.direBonjour = function () {  
    console.log('Bonjour');  
}  
exports.direBonsoir = () => {  
    console.log('Bonsoir')  
}  
}
```

```
var mod = require('./mesModules')  
mod.direBonjour()  
mod.direBonsoir()
```



Exemples

- **Exemple 3 : On n'est pas obligé de tout exporter**

- Au niveau du fichier : **mesModules.js**

```
var direBonjour = function () {  
    console.log('Bonjour');  
}  
  
var direBonsoir = () => {  
    console.log('Bonsoir')  
}  
  
module.exports = {bonjour : direBonjour}
```

- Au niveau du fichier : **test.js**

```
var mod = require('./mesModules')  
mod.bonjour()
```



Les fonctions callback

- **C'est quoi une fonction callback ?**

- Une fonction de rappel (en français) est une fonction passée dans une autre fonction en tant qu'argument, qui est ensuite invoquée à l'intérieur de la fonction externe pour accomplir une sorte de routine ou d'action.

- **Convention :**

- Les fonctions callback prennent deux paramètres :
 - ❑ un paramètre **err** : une instance de la classe Error – si l'opération a échoué – ou null (si la fonction a bien été exécutée);
 - ❑ un paramètre **result** qui contient le résultat si la fonction n'a pas détecté d'erreurs



Les fonctions callback

- **Remarque :**

- En NodeJS, on utilise très souvent les callback surtout pour les traitements asynchrones.
- Les fonctions callbacks en NodeJS ont une signature similaire.

- **Exemple 1 :**

```
function salutation(name) {  
    console.log('Bonjour ' + name);  
}  
  
function bonjour(name, callback) {  
    name ? callback(name) : callback('inconnu')  
}  
  
bonjour('Anis', salutation);
```



Les fonctions callback

- Exemple 2 :

```
const division = (a, b, callback) => {  
  if (b !== 0) {  
    return callback(null, a / b);  
  }  
  return callback('Problème de division par zéro', null);  
};  
const a = 10, b = 2;  
division(a, b, (err, result) => {  
  if (err) {  
    console.error("erreur : " + err);  
  } else {  
    console.log(`${a} / ${b} = ${result}`);  
  }  
});
```



Les fonctions callback

- **Exercice :**

- Considérons les données suivantes :

- `var tab = [1,3,6,8,9];`
- `var element = 6;`

- Ecrire une fonction qui permet de vérifier si `element` appartient à `tab`

- Démarche :

- ☐ Mettre les données dans un fichier : ***data.js***
- ☐ Définir la fonction callback `searchElement` afin d'afficher la position de l'élément dans le tableau sinon afficher un message d'erreur : ***searchElement.js***
- ☐ Faites l'appel de la fonction callback appliquée aux données définies dans le fichier : ***main.js***



Les fonctions callback

- Corrigé de l'exercice :

- Fichier *data.js* :

```
var tab = [1, 3, 6, 8, 9];  
var element = 6;  
  
exports.data = { tableau: tab, filtre: element };
```

- Fichier *searchElement.js* :

```
exports.searchElement = function (data, callback) {  
    for (var i = 0; i < data.tableau.length; i++)  
        if (data.tableau[i] == data.filtre)  
            return callback(null, i);  
    return callback('Element ' + data.filtre + ' non retrouvé dans le tableau');  
};
```



Les fonctions callback

- Corrigé de l'exercice :

- Fichier *main.js* :

```
var {searchElement} = require('./searchElement')
var {data} = require('./data')

searchElement(data, function (err, result) {
  if (err)
    console.error("erreur :" + err)
  else
    console.log(data.filtre + " existe a la position " + result)
});
```



Les fonctions synchrones et asynchrones

- **Les fonctions synchrones :**

- Par défaut, toute fonction définie en JavaScript est synchrone :
 - cette fonction exécute immédiatement l'intégralité de ses instructions puis retourne une valeur dans la foulée;
 - et que le reste du programme attend la fin de l'exécution de cette fonction avant de s'exécuter à son tour.

```
// console.log() est une fonction synchrone  
console.log('a');  
console.log('b');  
console.log('c');  
// => les lettres a, b et c seront systématiquement affichées dans l'ordre
```



Les fonctions synchrones et asynchrones

- **Les fonctions asynchrones :**

- permettre l'exécution de plusieurs opérations en parallèle sans bloquer l'exécution du reste du programme.

```
// setTimeout() : fonction asynchrone qui exécute la fonction de callback  
setTimeout(() => console.log('a'), 50); // afficher a dans 50 ms  
setTimeout(() => console.log('b'), 90); // afficher b dans 90 ms  
setTimeout(() => console.log('c'), 20); // afficher c dans 20 ms  
// => ordre d'affichage: c, a, puis b car l'exécution se fait en parallèle
```

- JavaScript fournit plusieurs manières pour définir et appeler des fonctions asynchrones :
 - Les fonctions de callback
 - Le principe des promesses : promise
 - L'usage de async et await



Les fonctions synchrones et asynchrones

Exemple 1 :

- **Fichier salutation.txt :**

bonjour
bonsoir
salut
aurevoir

- **Fichier synch.js :**

```
var fs= require('fs');  
var content =  
fs.readFileSync('./salutation.txt');  
console.log(content.toString());  
console.log('end of file');
```

- **Fichier asynch.js :**

```
var fs = require('fs');  
var content = fs.readFile('./salutation.txt',  
    function (err, result) {  
        if (err)  
            console.error(err);  
        console.log(result.toString());  
    });  
console.log('end of file');
```



Les fonctions synchrones et asynchrones

- **Exemple 2 :**

- Ecrire un programme NodeJS qui permet de créer un répertoire monDossier et trois fichiers file1.txt, file2.txt et file3.txt qui seront situés dans monDossier

```
var fs = require("fs");
var file = ["file1.txt", "file2.txt", "file3.txt"];
if (fs.existsSync('monDossier'))
    console.error('dossier existe deja');
else
    fs.mkdirSync('monDossier');
for (let i = 0; i < file.length; i++) {
    fs.writeFile('monDossier/' + file[i], 'contenu fichier', (err) => {
        if (err)
            console.error(err);
    });
}
```



Node JS : Création de serveur

- Création d'un serveur :

```
/* Inclure Le module interne http pour la création du serveur HTTP */
const http = require('http');

/* Création du serveur HTTP */
const server = http.createServer(function (request, response) {
    response.writeHead(200); /* Définir Les entêtes de La réponse HTTP*/
    response.end('Hello World!'); /* Envoyer Le contenu html dans Le corps de La
réponse HTTP */
});
/* Démarrer Le serveur HTTP en écoutant Le port 4000 */
/* Exécuter une fonction pendant que Le serveur est à l'écoute */
server.listen(4000, () =>
    console.log('Adresse du serveur : http://localhost:4000'));

```

- pour tester, lancer la commande **node app.js** et aller à l'URL
localhost:4000



Node JS : Création de serveur

- Création d'un serveur : une autre façon d'écrire

```
const {createServer} = require('http');

const server = createServer((request, response) => {
  response.writeHead(200, {'Content-Type': 'text/plain' });
  response.end('Hello World!');
});

server.listen(4000, () =>
  console.log('Adresse du serveur : http://localhost:4000'));;
```



Node JS : Création de serveur

- Retourner du code HTML

```
var {createServer} = require('http');
var server = createServer( (req, res) => {
  res.writeHead(200, { "Content-Type": "text/html" });
  res.write('<!DOCTYPE html>' +
    '<html>' +
    ' <head>' +
    ' <meta charset="utf-8" />' +
    ' <title>Ma page Node.js </title>' +
    ' </head>' +
    ' <body>' +
    ' <p>Hello world</p>' +
    ' </body>' + '</html>');
  res.end();
});
server.listen(4000, () =>
  console.log('Adresse du serveur : http://localhost:4000'));
```



Node JS : Création de serveur

- Le module url pour la récupération des informations de l'url

```
var { createServer } = require('http');
var url = require('url')
var server = createServer((req, res) => {
  var page = url.parse(req.url).pathname;
  console.log(page);
  res.writeHead(200, { "Content-Type": "text/html" });
  res.write('<h1>Hello world, this is your requested page : ' + page + '</h1>');
  res.end();
});
server.listen(4000, () =>
  console.log('Adresse du serveur : http://localhost:4000'));
```



Node JS : Création de serveur

- Le module `querystring` pour la récupération des paramètres de l'url

```
var { createServer } = require('http');
var url = require('url')
var querystring = require('querystring')
var server = createServer((req, res) => {
  var params = querystring.parse(url.parse(req.url).query);
  res.writeHead(200, { "Content-Type": "text/plain" });
  if ('prenom' in params && 'nom' in params) {
    res.write('Vous etes ' + params['prenom'] + ' ' + params['nom']);
  }
  else {
    res.write('Vous devez bien avoir un prenom et un nom, non ?');
  }
  res.end();
});
server.listen(4000, () =>
  console.log('Adresse du serveur : http://localhost:4000));
```



Node JS : Création de serveur

- **Exercice :**

- Ecrire un programme qui affiche le résultat d'une opération arithmétique (addition, soustraction, multiplication, division) des nombres passés en paramètre.

- **Exemples d'exécution :**

← → ↻ ⓘ localhost:4000/addition?x=5&y=9&z=8

Resultat : 22

← → ↻ ⓘ localhost:4000/multiplication?a=12&b=11

Resultat : 132



Node JS : Création de serveur

- Extraits du code source

:

```
function calcul(tab, operator) {  
  var result = '';  
  for (var i in tab) {  
    result = result +  
      operator + tab[i];  
  }  
  return eval(result.slice(1));  
}
```

```
var server = http.createServer(function (req, res) {  
  var pathname = url.parse(req.url).pathname;  
  var params =  
    querystring.parse(url.parse(req.url).query)  
  ;  
  res.writeHead(200, { 'Content-type': 'text/plain' });  
  var result;  
  if (pathname === '/addition') {  
    result = calcul(params, '+');  
  } else if (pathname === '/soustraction') {  
    result = calcul(params, '-');  
  } else if (pathname === '/multiplication') {  
    result = calcul(params, '*');  
  } else if (pathname === '/division') {  
    result = calcul(params, '/');  
  }  
  res.end('Resultat : ' + result);  
});
```

3.

Express JS

- ☐ Présentation générale
- ☐ Mise en place du serveur
- ☐ Gestion des routes
- ☐ Le package nodemon
- ☐ Les middlewares



Framework Express : présentation

- Quelques frameworks NodeJS :
 - **ExpressJS**
 - Ionic
 - NestJS, ...
- **Express.js (ExpressJS ou Express)** : micro-Framework pour Node.js.
 - Fournir des outils de base pour aller plus vite dans la création d'applications Node.js.
 - Offrir des fonctionnalités pour :
 - La gestion des routes (système de navigation)
 - Un moteur de Templates (Les vues de l'application)
 - Les middlewares : fonctions ayant accès à l'objet *request* et *response*



Mise en place d'un serveur express

- Installation :
 - `npm install --save express`
- Tester le démarrage du serveur :

```
const express = require('express')
const app = express()
const port = 4000

app.listen(port, () => {
  console.log(`Server is running on port ${port}`)
})
```



Gestion des routes

- Express.js : Gestion des routes simples

```
const express = require('express')
const app = express()
const port = 4000

app.get('/', function(req, res) {
  res.send('Vous êtes à la page d\'accueil du serveur');
});

app.listen(port, () => {
  console.log(`Server is running on port ${port}`)
})
```



Gestion des routes

- **Express.js** : Gestion des routes dynamiques

```
app.get('/infos/:code', function(req, res) {  
    res.setHeader('Content-Type', 'text/plain');  
    res.send('Vous êtes la personne ayant le code' + req.params.code);  
});
```

- Au niveau du serveur node JS, localhost:4000; en évitant de passer le suffixe “?variable=valeur”, les urls s’écrivent comme suit :

/infos/1
/infos/2
/infos/3

```
app.get('/infos/:code/:nom', function (req, res) {  
    res.send('Vous êtes la personne ayant le code : '+req.params.code +  
            "et le nom : "+req.params.nom);  
});
```



Gestion des routes avec Express

- Exemple :

```
const express=require("express");
const app=express();
```

```
app.get('/',(req,res)=>{
  res.setHeader('content-type','text/html')
  ; res.send('<h1>Hello Express</h1>');
```

Route statique



```
});
```

Route dynamique

```
app.get('/infos/:code',(req,res)=>{
  res.setHeader('content-type','application/json');
  var infos={name:'Express',email:'foulen@benFoulen.net',
  res.end(JSON.stringify(infos));
```

code:req.params.code};

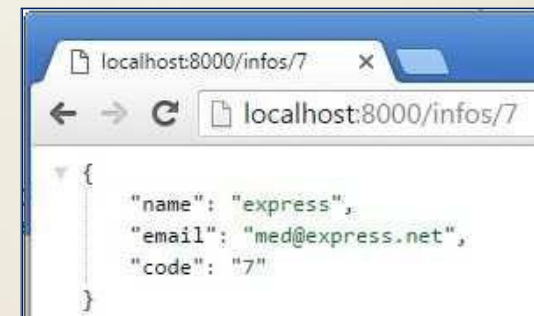
```
});
```

```
app.listen(4000,()=>{
  console.log('Server Started ..');
```

Serveur Node JS

```
});
```

```
C:\NodeJS\FirstAppNodeJS>node server-express
Server Started ..
```





Gestion des routes avec Express

- Réponse JSON :

- ❑ res.json()

```
res.end(JSON.stringify(infos));
```

- ❑ res.json(infos)

- Bon affichage des formats JSON :

- ❑ Installer l'extension : JSON Viewer
au niveau du navigateur

```
// 20220821103236
// http://localhost:4000/infos/1

{
  "name": "Express",
  "email": "foulen@benFoulen.net",
  "code": "1"
}
```





Package nodemon

- Pour éviter le redémarrage du serveur à chaque fois, vous pouvez installer :

- **Nodemon** : (<https://www.npmjs.com/package/nodemon>)

- > `npm install -g nodemon`
 - > `npm install --save-dev nodemon`

- Il surveille les modifications de fichiers sources
- Au niveau de "start " de " scripts ", mettez :

- ***nodemon*** à la place de "start "

```
> nodemon server.js
[nodemon] 2.0.12
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,json
[nodemon] starting `node server.js`
Server is running on port 4000
```

```
{
  "scripts": {
    "test": "echo \\\"Error: no test specified\\\" && exit 1",
    "start": "nodemon server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^4.17.1"
  },
  "devDependencies": {
    "nodemon": "^2.0.12"
  }
}
```



Exercice

- **Projet node :**
 - Créer un nouveau projet *node* avec le fichier d'entrée *app.js*
 - Installer *express* ainsi que *nodemon*
- **Source de données :**
 - Créer un nouveau fichier *users.js* incluant les utilisateurs trouvés sur cette API : <https://jsonplaceholder.typicode.com/users>
- **Routes :**
 - Créer la route : */users* afin d'afficher en format JSON la liste de tous les utilisateurs
 - Créer la route : */users/:id* affichant les informations d'un utilisateur dont on connaît l'id



Exercice

- Code source : Fichier *app.js*

```
const express = require("express");
const users= require('./users')
const app = express();

app.get('/users', (req, res) => {
  res.setHeader('content-type', 'application/json');
  res.json(users);
});
app.get('/users/:id', (req, res) => {
  res.setHeader('content-type', 'application/json');
  let id = parseInt(req.params.id)
  let user = users.find(u => u.id === id)
  res.json(user);
});
app.listen(4000, () => {
  console.log('Server Started ..');
});
```



Exercice

- Version améliorée du code source

:

Fichier *utils.js*

```
exports.success = (message, data) => {  
  return { message, data }  
}
```

Fichier *app.js*

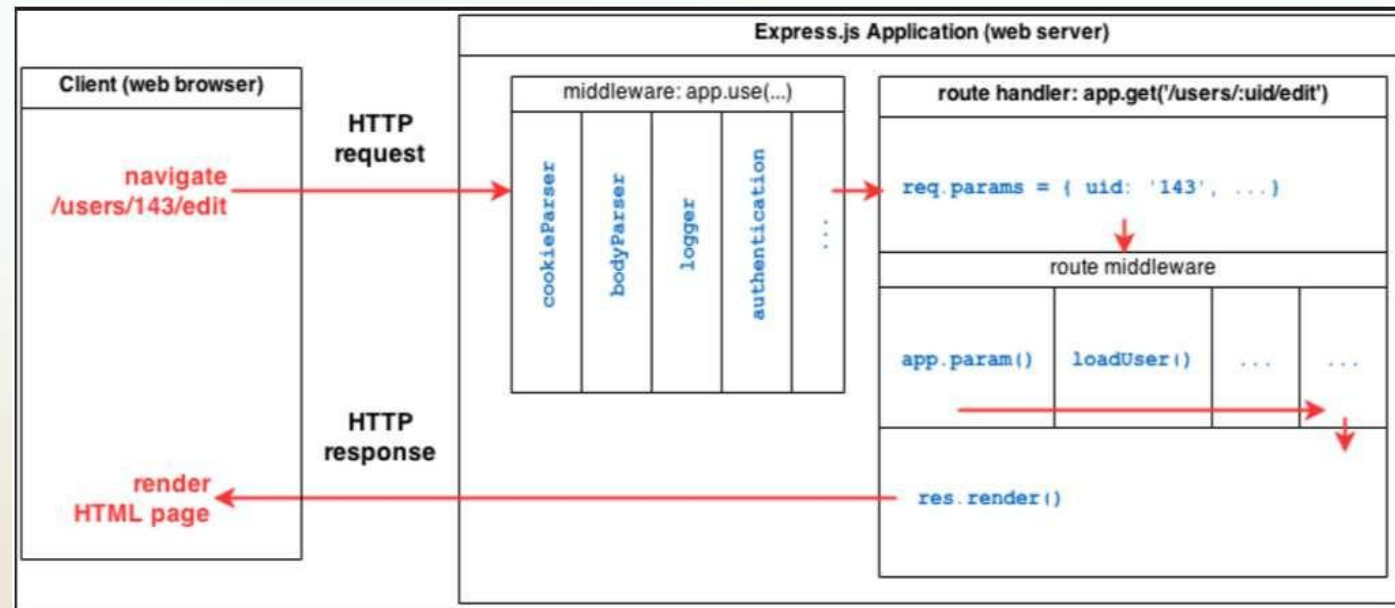
```
const express = require("express");  
const users = require('./users') const  
{success} = require('./utils') const  
app = express(); app.get('/users',  
(req, res) => {  
  res.setHeader('content-type', 'application/json');  
  let msg = 'La liste des users a été bien récupérée'  
  res.json(success(msg, users));  
});  
app.get('/users/:id', (req, res) => {  
  res.setHeader('content-type', 'application/json');  
  let id = parseInt(req.params.id)  
  let user = users.find(u => u.id === id)  
  let msg = "L'utilisateur a été bien récupéré"  
  res.json(success(msg, user));  
});  
app.listen(4000, () => {  
  console.log('Server Started ..');  
});
```



Framework Express : les middlewares

- **Express.js** : Framework basé sur le concept de middlewares

- Ce sont des petits morceaux d'application qui rendent chacun un service spécifique.
- Express est fourni avec une quinzaine de middlewares de base, et d'autres développeurs peuvent bien entendu en proposer d'autres via NPM.





Framework Express : les middlewares

- **Application ExpressJS** = { fonctions } appelées middlewares
- **Middleware** : Fonction ayant accès à l'objet request et response

- **Déclaration :**

```
const middleWare = (request, response, next)=> {  
  // traitement;  
  next();  
};
```

- **Utilisation :**

Il suffit d'appeler la méthode **app.use(middleware)** pour utiliser un middleware.

On peut enchaîner plusieurs **app.use("Mid1"), .use("Mid2"), use("Mid3")**



Framework Express : les middlewares

- Les middlewares sont interconnectés dans un pipe line et peuvent communiquer entre eux.
- Tous ces middlewares communiquent entre eux en se renvoyant jusqu'à 4 paramètres :
 - err : les erreurs
 - req : la requête du visiteur
 - res : la réponse à renvoyer (la page HTML et les informations d'en-tête)
 - next : un callback vers la prochaine fonction à appeler

Middleware 1 (err, req, res, next)



Middleware 2 (err, req, res, next)



Middleware 3 (err, req, res, next)



Framework Express : les middlewares

- Exemple :

```
const express = require('express');
const app = express();
const middleWare = (request, response, next) => {
  console.log("middleWare:", request.url);
  next();
};
app.use(middleWare);
app.get('/', (request, response, next) => {
  console.log('requête reçue');
  response.send('Hello World!');
  next();
});
app.listen(4000, () => console.log('Adresse du
serveur : http://localhost:4000'));
```

Adresse du serveur : <http://localhost:4000>
middleWare: /
requête reçue

Déplacer app.use(middleWare) après app.get □

Adresse du serveur : <http://localhost:4000>
requête reçue
middleWare: /

Supprimer next au niveau de app.get □

Adresse du serveur : <http://localhost:4000>
requête reçue



Framework Express : les middlewares

- Exemple : autre écriture

```
const express = require('express');
const app = express();
const middleWare = (request, response, next) => {
  console.log("middleWare:", request.url);
  next();
};
app.get('/', (request, response, next) => {
  console.log("requête reçue");
  response.send('Hello World!');
  next();
}, middleWare);
app.listen(4000, () => console.log('Adresse du serveur : http://localhost:4000'));
```

Adresse du serveur : <http://localhost:4000>
requête reçue
middleWare: /



Framework Express : les middlewares

- Qu'affiche le programme suivant (dans la console) ?

```
const express = require('express');
const app = express();
const middleware1 = (request, response, next) => {
  console.log("middleware 1");
  next();
};
const middleware2 = (request, response, next) => {
  console.log("middleware 2");
  next();
};
app.use([middleware2, middleware1]);
app.get('/', (request, response, next) => {
  response.send('Hello World!');
});
app.listen(4000, () => console.log('Adresse du serveur : http://localhost:4000'));
```

4.

Les bases de données NoSQL : MongoDB

- ☐ Présentation
- ☐ Installation
- ☐ MongoShell (opérations CRUD)
- ☐ MongoDB Compass
- ☐ MongoDB Atlas : persistance dans le cloud





Introduction

- **Bases de données NoSQL (Not Only SQL) :**
 - BD non relationnelles utilisées pour stocker et récupérer des données
 - Nouvelle façon de gérer les données : relâcher certaines contraintes lourdes du relationnel pour favoriser la distribution
 - Intensément utilisées dans le cadre d'applications Web en temps réel.
 - Appelées aussi bases de données **Big Data** ou bases de données **Cloud**
 - **Types de BD NoSQL :** BD de documents, magasins de valeurs clés, mémoires à colonnes larges, BD orientée graphes
 - **BDs NoSQL les plus connues :** MongoDB, Cassandra, Redis, Hbase, Neo4j, RavenDB, OracleNoSQL,



MongoDB : présentation

- **MongoDB :**

- Système de gestion de base de données **NoSQL** orientée documents
- Créé en 2007, Open-source, développé en C++
- Disponibilité de plusieurs fonctionnalités SQL (COUNT, GROUP BY, ORDER BY, SUM...)
- Possibilité d'accéder aux données via une console JavaScript
- Des drivers disponibles pour plusieurs langages de programmation (Java, JavaScript, PHP, Python, Ruby...)
- Données stockées sous format JSON (JavaScript Object Notation)
- SGBD NoSQL le plus populaire (utilisé par MTV, Disney, Doodle, Adobe, eBay ...)



MongoDB : présentation

- SQL Vs MongoDB :

- Base = **Base**
- Table = **Collection**
- Enregistrement (tuple) = **Document**
- En BDR, tous les tuples d'une table ont les mêmes champs (mais les valeurs peuvent être différentes (les valeurs sont affectées à des **colonnes**))
- Dans une collection **MongoDB**, les documents peuvent ne pas avoir un champ partagé (pas de colonnes dans un document MongoDB).

Stockage orienté lignes					Stockage orienté colonnes							
id	type	lieu	spec	intérêts	id	type	id	lieu	id	spec	id	intérêts
Nicolas	prof	CNAM	BDD, NoSQL	BZH, Star Wars	Nicolas	prof	Céline	Centrale Supelec	Nicolas	BDD	Nicolas	BZH
Régis		OC	Machine Learning, Dev	escalade, nouilles chinoises	Céline	prof	Nicolas	CNAM	Nicolas	NoSQL	Nicolas	Star Wars
Luc	resp formation OC	OC	formation, audiovisuel		Luc	resp formation OC	Régis	OC	Régis	Machine Learning	Régis	escalade
Céline	prof	CentraleSupelec	Ontologie, logique formelle, visualisation				Luc	OC	Régis	Dev	Régis	nouilles chinoises
									Luc	formation		
									Luc	audiovisuel		
									Céline	Ontologie		
									Céline	logique formelle		
									Céline	visualisation		



Installation

- **Téléchargement et mise en place :**
- Allez sur le lien :
 - <https://www.mongodb.com/try/download/community> (onglet Community server), choisissez la dernière version, msi comme Package et Windows comme Platform.
- Installez le fichier téléchargé.
- Sous la racine du disque dur (C:\ sous Windows), créez l'arborescence data\db : c'est l'arborescence par défaut qui sera cherchée par MongoDB.
- **Documentation officielle de MongoDB :** <https://docs.mongodb.com/>



Installation

- **Ajouter MongoDB au path de Windows :**
- Copiez le chemin absolu du répertoire **bin** de MongoDB dans "Programmes files"
- Dans la barre de recherche, cherchez "**Systeme**" ensuite cliquez "**Paramètres système avancés**"
- Choisissez "**Variables d'environnement**" ensuite dans "**Variables utilisateur**" cliquez sur "**Nouvelle**"
- Saisissez comme nom de variable PATH et comme valeur le chemin absolu du répertoire bin de MongoDB dans "Programmes files"



MongoShell : Connexion et création d'une base de données

- Démarrage du serveur MongoDB : `> mongod`

- Connexion à MongoDB Shell : `> mongo`

connecting to: mongod://127.0.0.1:27017/ (num de port par défaut : 27017)

- Après connexion : `use nomBD`
- En se connectant : `mongo --port numeroPort nomBD`
- Création :
 - Si la base de données n'existe pas, elle sera créée.
- Exemple de création et d'utilisation d'une base de données **ma base** : `use ma_base`



Connexion et création d'une base de données

- Lister les bases de données existantes :
 - `show dbs`
- Par défaut :
- Il existe trois bases de données :
 - **admin, config et local**
- Si on ne se connecte pas à une base de données, on utilise par défaut une base de données appelée : `test`



Suppression d'une base de données

- Suppression de la base de données courante
:
- Après connexion : `use nomBD`
 - `db.dropDatabase()`
 - `db.runCommand({dropDatabase: 1})`



Gestion des collections

- **Collection :**

- **Une collection :** est l'équivalent d'une table en SQL.

- **Création d'une collection :**

- Deux solutions :

- Directement : `db.createCollection('personne')`
- En insérant un document :

```
db.personne.insert({ nom:"Assas", prenom: "Anis" })
```



Gestion des collections

- Lister les collections existantes :
 - `show collections`, ou
 - `show tables`, ou
 - `db.getCollectionNames()`
- Suppression d'une collection :
 - `db.nomCollection.drop()`



Gestion des documents : Insertion

- **Document :**

- **Un document** : est l'équivalent d'un tuple en SQL.
- Chaque document possède un **_id** attribué par l'utilisateur ou par MongoDB (ObjectId). Le champ **_id** constitue l'index de la collection.

- **Ajout d'un document** : Insertion simple ou multiple

- `db.nomCollection.insert({ clé1:'val1', clé2:'val2' ... })`
- ou bien `obj=({ clé1 : 'val1', clé2 : 'val2' ... })` ensuite
- `db.nomCollection.save(obj)` (**save** ajoute ou modifie un document)



Gestion des documents : Insertion

- Exemple :

```
db.personne.insert({  
  nom: 'assas',  
  prenom: 'anis',  
  age: 43, sportif:  
  true  
})
```

- Ou encore avec save :

```
db.personne.save({  
  nom: 'assas',  
  prenom: 'anis',  
  age: 43,  
  sportif: true  
})
```

```
objet = {  
  _id: 10, nom:  
  'assas',  
  prenom: 'anis',  
  age: 45,  
  sportif: true  
}
```

```
db.personne.save(objet);
```



Gestion des documents : Insertion

- Exemple d'insertion multiple :

```
db.personne.insert( [  
  {  
    nom: 'ben ahmed',  
    prenom: 'ahmed',  
    age: 43, sportif:  
    true  
  },  
  {  
    _id: 12,  
    nom: 'ben salah',  
    prenom: 'salah',  
    niveau: 'master',  
  }  
] )
```

- Pour vérifier que l'ajout a eu lieu :

```
db.nomCollection.find()
```



Gestion des documents : Insertion

- Autres instructions d'insertion :

- `db.collection.insertOne()` New in version 3.2

```
db.inventory.insertOne(  
  { item: "canvas", qty: 100, tags: ["cotton"], size: { h: 28, w: 35.5, uom: "cm" } }  
)
```

- `db.collection.insertMany()` New in version 3.2

```
db.inventory.insertMany([  
  { item: "journal", qty: 25, tags: ["blank", "red"], size: { h: 14, w: 21, uom: "cm" } },  
  { item: "mat", qty: 85, tags: ["gray"], size: { h: 27.9, w: 35.5, uom: "cm" } },  
  { item: "mousepad", qty: 25, tags: ["gel", "blue"], size: { h: 19, w: 22.85, uom: "cm" } }  
)
```



Gestion des documents : Lecture

- Récupérer tous les documents d'une collection : **find()**
 - Exemple : `db.personne.find()`
- Rechercher selon des critères :
 - Exemple : `db.personne.find({nom: "assas"})`
- Limiter le nombre de documents à afficher : **limit()**
 - Exemple : `db.personne.find().limit(2)`
`db.personne.find({nom: "assas"}).count()`
- Trier le résultat de recherche par ordre croissant : **sort()**
 - `db.personne.find().sort({nom : 1})` // Ordre décroissant `{nom : -1}` 75



Gestion des documents : Lecture

- **Autres utilisations de find() :**

- Afficher le nom des personnes : `db.personne.find({}, {nom:1})`
- Sans affichage de l'identifiant : `db.personne.find({}, {nom:1, _id:false})`
- Appel d'une fonction personnalisée :

```
db.personne.find().forEach(  
    function(perso){  
        print(perso.nom + " " + perso.prenom);  
    });
```
- Afficher seulement le premier document : `db.personne.findOne()`
- Afficher le résultat en format json :
`db.personne.find().pretty();`



Gestion des documents : Lecture

- Utilisation des expressions régulières :
 - Le nom commence par w : `db.personne.find({nom:/^w/})`
 - Le nom se termine par w : `db.personne.find({nom:/w$/})`
 - Le nom commence par e ou par h : `db.personne.find({name:/^[eh]/})`
 - Commence par lettre entre e et w : `db.personne.find({name:/^[e-w]/})`
 - On pourrait également utiliser regex :
 - `db.employees.find({ prenom: { $regex: /john/} })`



Gestion des documents : Modification

- **Modification d'un document :**

- **update()** : modifier un ou plusieurs documents selon une ou plusieurs conditions.
- **save()** : remplacer toutes les valeurs d'un document selon l'identifiant par les valeurs indiquées dans la méthode. Si l'identifiant n'existe pas il sera ajouté.
- **updateOne()** : modifier uniquement le premier document de la collection (par défaut).
- **updateMany()** : modifier plusieurs documents.
- **replaceOne()** : remplacer le premier élément de la sélection.



Gestion des documents : Modification

- Exemple :

- ```
db.personne.update(
 {
 _id: 12
 },
 {
 prenom: 'Salah',
 }
)
```

- Le document ayant l'identifiant 12 a désormais un seul champ (hormis l'identifiant) : ***prenom***. Les autres champs définis au préalable ont été supprimés.



## Gestion des documents : Suppression

- **Suppression d'un document :**
  - **remove()** : supprimer un ou plusieurs documents selon une ou plusieurs conditions.
  - **deleteOne()** : supprimer uniquement le premier document de la collection
  - **deleteMany()** : supprimer plusieurs documents
- **Exemple :**
  - **db.personne.remove({nom: "bob"})** : Supprimer tous les documents dont le champ **nom** contient comme valeur **bob**.



# MongoDB Compass

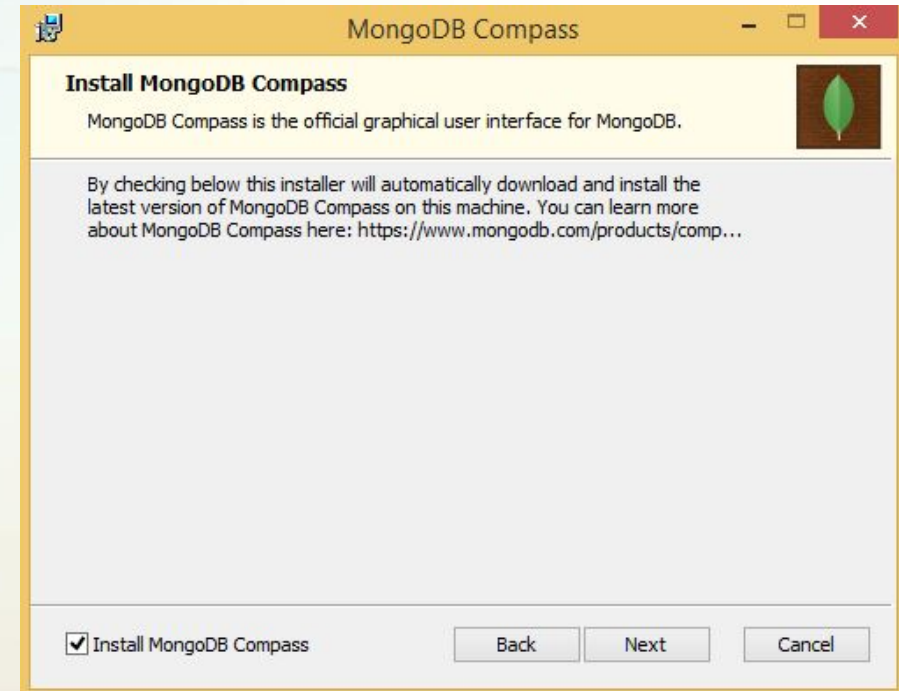
- **MongoDB Compass Community :**

La GUI (interface graphique utilisateur)

- Exploration visuelle des données.
- Exécution de requêtes ad hoc en quelques secondes.
- Interactions avec les données à l'aide de fonctionnalités CRUD complètes.

- **Installation :**

- <https://www.mongodb.com/try/download/compass>



**Ou bien tout simplement :** Cocher l'option "Install MongoDB Compass" lors de l'installation de « MongoDB Community Server »



# MongoDB Compass

- Connexion :
  - Cliquer sur l'onglet "Connect" → "Connect to" ou le raccourci "New Connection"
- Bases de données existantes :

The screenshot shows the MongoDB Compass interface. On the left, a sidebar labeled 'Local' displays '3 DBS' and '1 COLLECTIONS'. Below this, it shows the host 'localhost:27017', the cluster type 'Standalone', and the edition 'MongoDB 4.2.15 Community'. A search bar 'Filter your data' is present. A list of databases is shown: 'admin', 'config', 'local', 'ma-base', and 'personne'. A table titled 'Databases' is highlighted, showing the following data:

| Database Name | Storage Size | Collections | Indexes |
|---------------|--------------|-------------|---------|
| admin         | 20.0KB       | 0           | 1       |
| config        | 36.0KB       | 0           | 2       |
| local         | 44.0KB       | 1           | 1       |
| ma-base       | 20.0KB       | 1           | 1       |

The 'New Connection' dialog is shown with the 'Hostname' tab selected. It includes a 'Favorite' button and a 'Paste connection string' link. The form fields are as follows:

- Hostname: localhost
- Port: 27017
- SRV Record: ☐
- Authentication: None

A green 'Connect' button is located at the bottom right of the dialog.

Click edit to modify your connection string (SRV or Standard ⓘ)

```
mongodb://localhost:27017/?readPreference=primary&appName=MongoDB%20Compass&ssl=false
```

Edit Connect

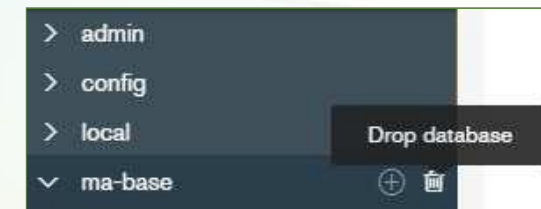
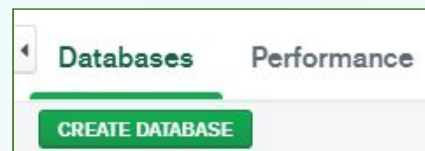


# MongoDB Compass

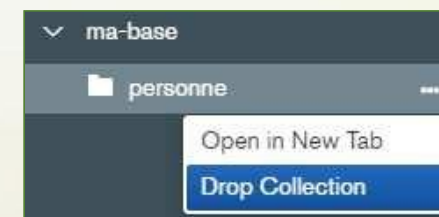
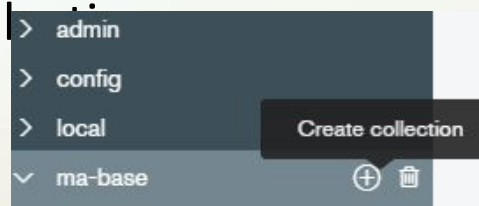
- Quelques opérations de manipulations de BD

:

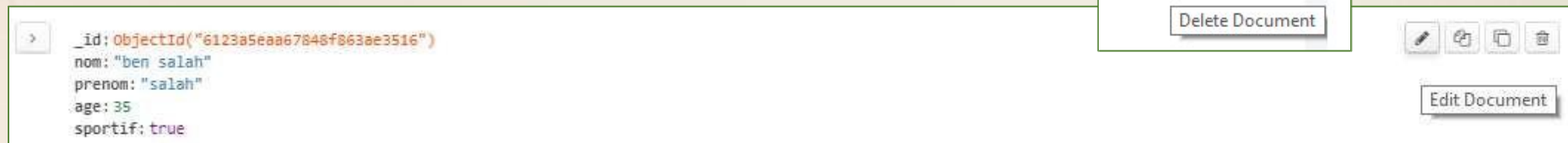
- Créer ou supprimer une BD :



- Créer ou supprimer une collection :



- Modifier ou supprimer un document





# MongoDB Atlas

- **C'est quoi MongoDB Atlas ?**

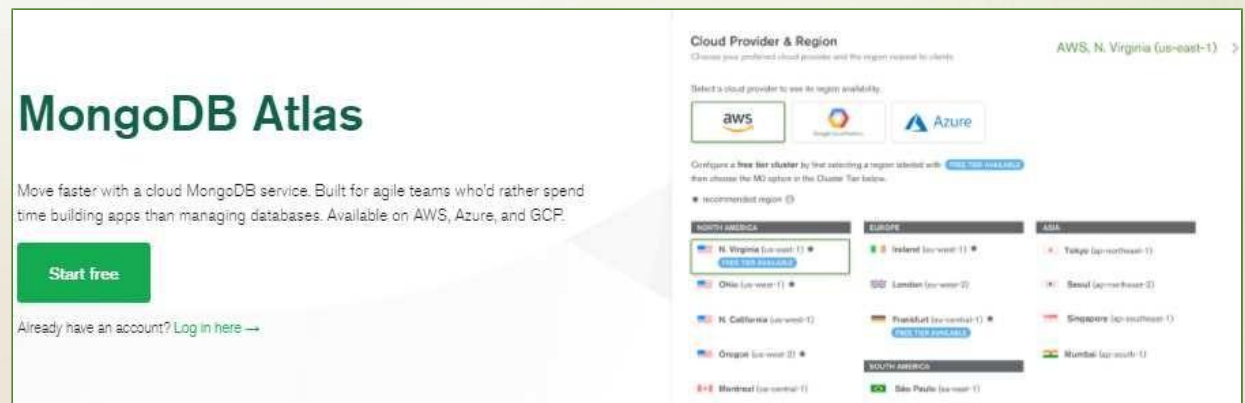
- BD documentaire flexible et évolutive s'inscrivant dans le domaine de la base de données en services (« Database as a Service » ou DBaaS).
- BD cloud basée sur le serveur NoSQL open source MongoDB.
- Permet de créer un cluster de BD multicloud sur AWS, Microsoft Azure et Google Cloud

## Lien :

<https://www.mongodb.com/fr-fr/cloud/atlas>

### > Start free

- Vous vous connectez par votre compte (si vous êtes déjà inscrit) ou via votre compte google.



5.

Exemple d'application FullStack



## Atelier : activité pratique

### ❑ Développement d'une application web fullStack MERN

- Démarche :
- Création de la base de données et persistance MongoDB Atlas dans le cloud
- Développement de l'API avec Express JS (CRUD)
- Tester les web services (routes) avec l'outil postman
- Bien structurer le code et sécuriser les API's
- Interaction du serveur avec l'application Client ReactJS



## Activité pratique

- Architecture de l'application :

